FreshMart Sales Analytics — Phase 1: Data Cleaning Report

Objective

Before starting any exploratory analysis, I wanted to make sure the raw data could actually be trusted. This report summarises the issues I found in each table, the decisions I made to fix them, and why I made those decisions — since I think *why* matters as much as *what* in a cleaning process.

My guiding principle

I decided early on not to impute anything I couldn't reasonably infer — especially IDs, dates, and transaction quantities. A wrong guess in those fields is worse than an honest gap, since it can quietly distort every chart built on top of it later. I only filled in values where a safe, defensible estimate existed (like using the median for cost/weight fields, since it isn't pulled around by outliers the way a mean would be).

Table-by-table summary

Table	Rows	Issues found	What I did
Calendar	730	Dates stored in 5 different formats	Wrote a custom parser instead of pd.to_datetime(format="mixed") — see "key challenge" below. 0 duplicate dates remained after the fix.
Customers	10,281	Inconsistent text in country, marital status, gender, homeowner, income bracket; mixed casing in names/city/education; missing values in postal code (611), birthdate (515), marital status (184), income (1,044), account-open date (105), occupation (841)	Standardized all categorical text to one consistent spelling/casing; converted income into an ordered category; parsed dates with the custom parser; **kept missing values as NaN** rather than guessing someone's income or birthdate
Products	1,560	Price column mixed "$10.13" strings with plain numbers; boolean flags (recyclable, low_fat) stored as 8 different representations (1/0/Y/N/Yes/No/True/False); missing cost (53), weight (60), recyclable (27), low_fat (26)	Converted price to numeric; standardized both flag columns to Yes/No; filled cost & weight with the **median** (outlier-safe); filled the two flag columns with the mode
Regions	109	Inconsistent casing in district/region names	Standardized casing; confirmed 0 duplicate region IDs

Table	Rows	Issues found	What I did
Returns	7,087 → 6,881	Mixed date formats; missing return_date (77); missing quantity (206)	Parsed dates with custom parser; **dropped the 206 rows missing quantity** since return quantity is the core metric and can't be inferred; kept the remaining 73 rows with missing dates since product/store info is still usable
Stores	24	store_type had 3 different spellings for the same categories (e.g. "supermarket" / "SUPERMARKET" / "Midsize Grocery"); country had 4 spellings; phone numbers in 4 different formats; missing phone (1), last remodel date (12), grocery sqft (3)	Standardized store type to 5 clean categories and country to "USA"; stripped phone numbers to digits only; filled grocery sqft with median; kept missing phone/remodel date as-is (unknowable)
Transactions 1997	86,837	Mixed date formats; missing transaction_date (415), stock_date (895), customer_id (4,390), quantity (1,774)	Parsed dates with custom parser; kept all missing values — customer_id nulls represent real anonymous/cash sales, not errors, so I did not drop or fill them
Transactions 1998	182,883	Same pattern as 1997: missing transaction_date (912), stock_date (1,840), customer_id (9,097), quantity (3,587)	Same approach as 1997 for consistency

Tools used for review

I did all the cleaning decisions and code myself in pandas. Once I thought Phase 1 was done, I also used **Claude** as a second-pass reviewer — going through the notebook the way a second set of eyes would in a code review, specifically to check for anything that could quietly break downstream analysis. That review surfaced two real, technical issues I'd missed on my own, described below. I verified both independently before applying the fixes.

Key challenge: the date-parsing bug

My first pass used pd.to_datetime(col, format="mixed") everywhere, which ran without errors but was **silently wrong**. Since the dataset mixes month-first dates (04/21/1994) and day-first dates (26-07-1997), pandas' default assumption (dayfirst=False) misread some day-first dates — e.g. "04-06-1997" (meant to be June 4th) was being read as April 6th. This only affects dates where both day and month are ≤12, so it never throws an error — it just quietly produces a wrong date. This was flagged during the Claude review, which demonstrated the misparse by testing a known ambiguous value against both interpretations side by side. Once I understood the cause, I fixed it by writing a small function that matches each date string to its exact format pattern (by separator and structure) before parsing, instead of letting pandas guess — and applied it across every date column in every table for consistency.

A second issue: customer_id silently losing its missing values

After my first cleaning pass, I had converted customer_id to text using .astype(str) for consistency with other ID columns — but this silently turned missing values into the literal text "nan" instead of leaving them as real nulls, which would have made every anonymous transaction look like a "known" customer. The Claude review caught this by comparing the null count before and after the conversion. I fixed it using .astype("Int64").astype("string"), which preserves true missing values through the type conversion, and confirmed the null counts matched the original figures afterward. I also made sure product_id used the same data type (string) across every table it appears in, since a mismatched type between two tables would silently break a join later, even if each table looks clean on its own.

What's next

With all 8 tables cleaned, consistently typed, and every decision documented, I'm moving into the EDA phase — starting with monthly sales trends, customer segmentation by loyalty tier, and product-level return rates.

One thing I deliberately left untouched in this phase: outliers. I didn't flag or remove any during cleaning, since whether a value counts as a genuine outlier depends on the business context, not just the distribution — a large quantity could be a real bulk order, not an error. I'll identify and treat outliers during EDA, based on what the business questions actually require.